第5讲 数据在计算机内的表示形式

核心内容

- ✓ 数值数据在计算机内的表示形式
- ✓ 字符数据在机内的表示形式
- ✓ 数据类型转换
- ✓ 浮点数据在机内的表示形式

学习重点

有符号整数和无符号整数的表示与存储

字符串的表示方法与存储

浮点数据的表示方法与存储

地址类型转换 与 数据类型转换

课堂目标

- 给定一个整数，能正确给出其补码表示形式及存储结果
- 给定一个浮点数，能正确给出其表示形式及存储结果
- 给定一个字符串，能正确给出其表示形式及存储结果
- 给定一个结构数组，能正确给出其存储结果

对于 C 语句中定义的整型、浮点型、字符数组、结构数组、指针等能正确画出内存中存储的示意图；对于内存中看到的存储结果，能够写出对应的 C语言赋值语句。

对不同长度的整型变量之间的赋值，能够给出赋值结果。

- 数据
  - 数值数据
    - 定点表示法
    - 浮点表示法（实数）
  - 字符数据

常用的数值数据为二进制、十进制、十六进制、BCD码。

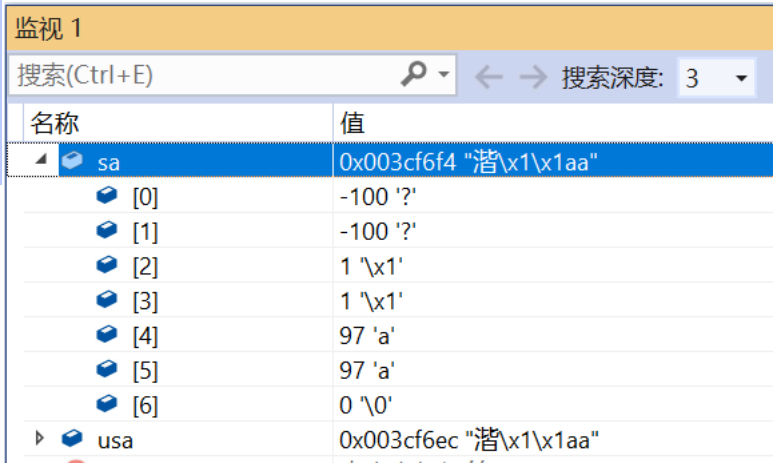
字符数据：ASCII（美国信息标准交换代码）

GB2312

GBK

2.1 有符号和无符号整数的表示形式

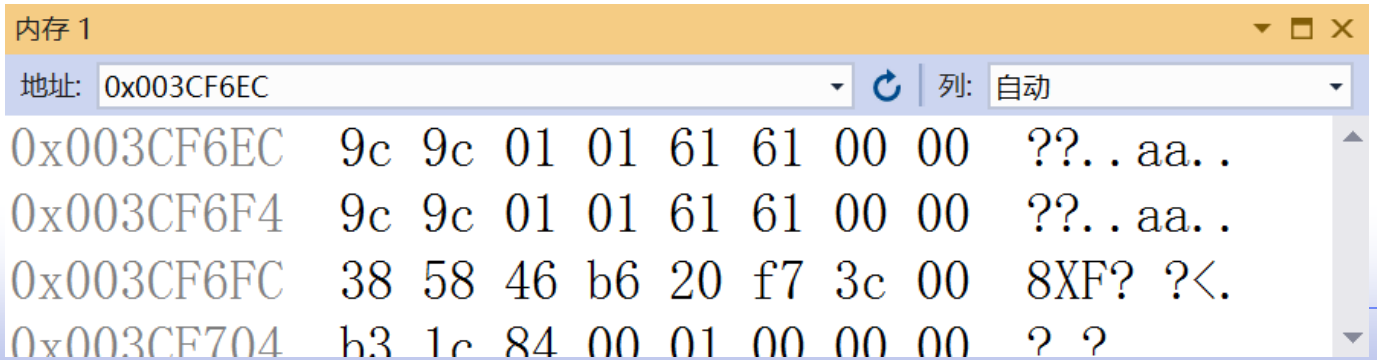
Q: 设 char sa[5] = {-100, 156, -255, 1, 'a', 97, 0};
unsigned char usa[5]= {-100, 156, -255, 1,'a',97, 0};
数组sa, usa中存放的结果是什么？为什么？



数据的存放与unsigned 无关。
但是相同字节流解读成有符号或者无符号数，是不同的。

sa[0] = 0x9c > 0 不成立

usa[0] = 0x9c > 0 成立！



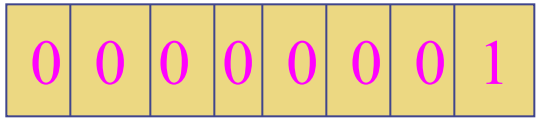
2.1 有符号和无符号整数的表示形式

■ 有符号数的表示范围

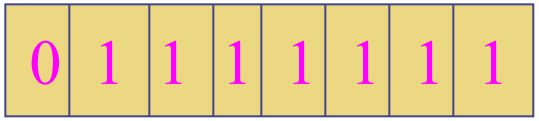
8位补码： 80H --- 7FH (-128 ,127)

16位补码： 8000H --- 7FFFH (-32768 , 32767)

正数



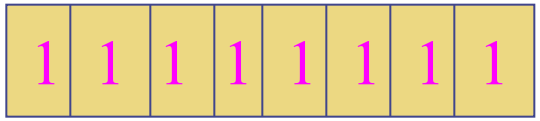
1



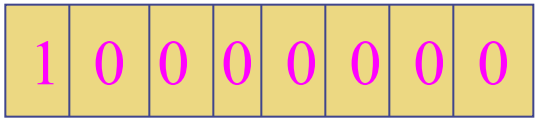
7F

N=8

负数



-1



-128

2.1 有符号和无符号整数的表示形式

- 有符号数（补码表示）的大小比较

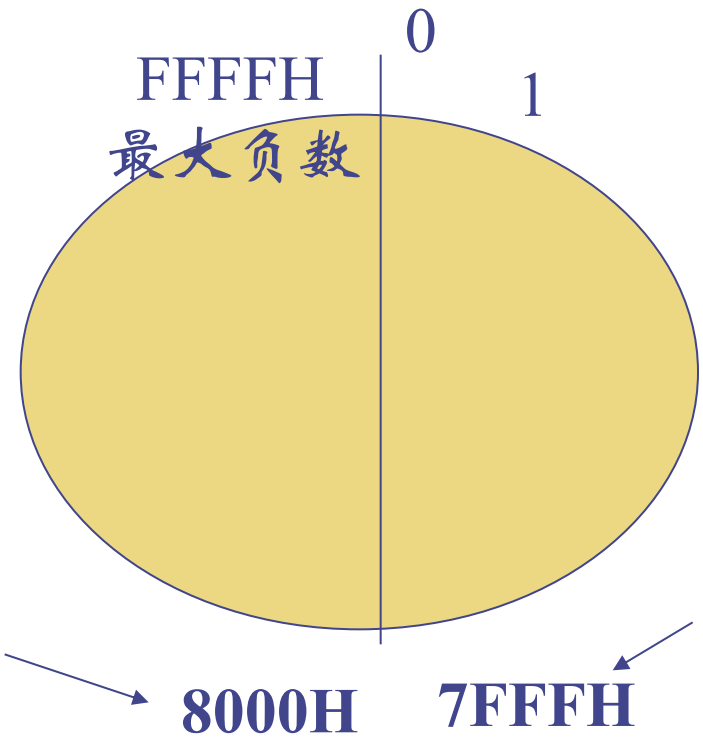
设 n = 16, 试比较：

5678H	vs	7345H ?	<
5678H	vs	8345H ?	>
3271H	vs	0A521H ?	>
0A521H	vs	0B521H ?	<

2.1 有符号和无符号整数的表示形式

- 有符号数（补码表示）的大小比较

5678H < 7345H
5678H > 8345H
3271H > 0A521H
0A521H < 0B521H



2.1 有符号和无符号整数的表示形式

- 无符号数及其表示范围

一个段中的偏移地址，就是一个无符号数。
最小的无符号数是0。

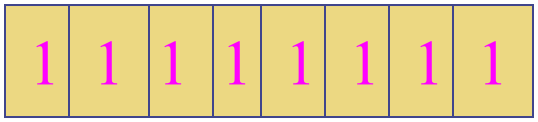
8位无符号数：　0H --- FFH　(0 ,255)

16位无符号数：　0H --- FFFFH　(0 ,65535)

无符号数的大小比较？

2.1 有符号和无符号整数的表示形式

设 n=8, 有一个数 M,



Question: 若将其看成一个数的补码，它表示的什么数？

-1

Question: 如果M作为无符号数，它表示的是多少？

255

Question: 哪到底将它看成什么数呢？

Answer: 这取决于访问该单元的指令。

有 / 无 unsigned 的变量, 编译生成的机器指令不同。

2.1 有符号和无符号整数的表示形式


```
#include <stdio.h>

void main( )

{
      short   x;
      unsigned short  y;
      x= -1;            // x=65535
      y= -1;            // y=65535
      printf("%d   %d\n", x,  y);

}
```

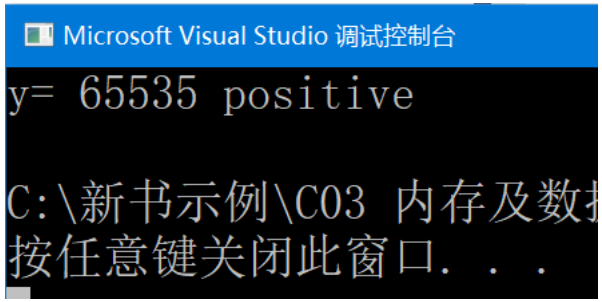
结果是：
-1   65535

在汇编语言中也有类似于unsigned的约定。

2.1 有符号和无符号整数的表示形式


```
#include <stdio.h>
void main( )
{
    short   x;
    unsigned short  y;
    x = -1;    y = -1;
    if (x > 0)
        printf("x= %d positive\n",x);
    if (y > 0)
        printf("y= %d positive\n",y);
}
```

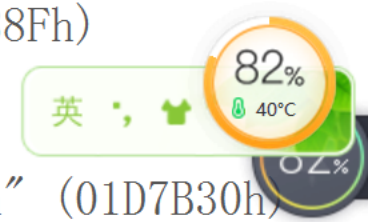
结果？



谁来把一个数当有符号数看，还是无符号数？

2.1 有符号和无符号整数的表示形式


```
    if (x > 0)  printf("x= %d positive\n",x);
001D1875  movsx       eax,word ptr [x]
001D1879  test        eax,eax
001D187B  jle         __$EncStackInitStart+43h (01D188Fh)
001D187D  movsx       eax,word ptr [x]
001D1881  push        eax
001D1882  push        offset string "x= %d positive\n" (01D7B30h)
001D1887  call        _printf (01D10CDh)
001D188C  add         esp,8
    if (y > 0)  printf("y= %d positive\n", y);
001D188F  movzx       eax,word ptr [y]
001D1893  test        eax,eax
001D1895  jle         __$EncStackInitStart+5Dh (01D18A9h)
001D1897  movzx       eax,word ptr [y]
001D189B  push        eax
001D189C  push        offset string "y= %d positive\n" (01D7B44h)
001D18A1  call        _printf (01D10CDh)
001D18A6  add         esp,8
```



小结

- 整型数据的表示与unsigned 无关。
- 不论是有符号整数还是无符号整数，都是采用补码表示。
- 同一个数字，当成有符号数解读与当成无符号数解读，解读的结果可能不同。
- C 语言中，char/short/int、unsigned char/short/int 是不同的类型；对于比较运算，生成的机器指令不同。

2.2 BCD码

BCD (Binary Coded Decimal):二进制编码的十进制

用4位二进制数表示一位十进制数。

1 = 0001 BCD　　8 = 1000 BCD

2 = 0010 BCD　　9 = 1001 BCD

……

9781的压缩BCD码

1000　0001
1001　0111

0000　0001
0000　1000
0000　0111
0000　1001

9781的非压缩BCD码

BCD码 扩展知识

BCD码可分为两类：有权BCD码和无权BCD码

有权BCD码：8421码、2421码、5421码

无权BCD码：余3码、余3循环码、格雷码

- 8421码：从高位到低位的权值为8、4、2、1，和四位自然二进制码不同的是，它只选用了四位二进制码中前10组代码，即用0000~1001分别代表它所对应的十进制数，余下的六组代码1010~1111不用。
- 2421码：从高位到低位的权值为2、4、2、1，注意：数码6，既可以用1100表示，也可以用0110表示
- 5421码：从高位到低位的权值分别为5、4、2、1，注意：数码5，既可以用1000表示，也可以用0101表示
- 余3码：在8421 BCD码的每基础上加3(0011)形成的，比如8->1011,9->1100
- 余3循环码：余3循环码是变权码，每一位的1并不代表固定的数值，它具有格雷码的优点，即两个相邻代码之间仅有一位的状态不同。
- 格雷码：在一组数的编码中，若任意两个相邻的代码只有一位二进制数不同，则称这种编码为格雷码（Gray Code），另外由于最大数与最小数之间也仅一位数不同，即“首尾相连”，因此又称循环码或反射码。

格雷码 Gray Code

十进制数	自然二进制数	格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

二进制码->格雷码（编码）：

- 最左边一位不变；
- 从左位第二位起，依次将每一位与左边一位异或（XOR），作为对应格雷码在该位的值。

- 相邻的数字编码，只有一个二进制位发生变化

- 常用于模拟－数字转换中

余3码

十进制数	8421码	余3码
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

以8421码（即BCD）为基础，
每位十进制数BCD码，加上0011得到。

2.3 字符数据在机内的表示形式

華中科技大學

ASCII码字符表

		0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
↓		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0				**0**		**P**		**p**								
1	1				**1**	**A**	**Q**	**a**	**q**								
2	2				**2**	**B**	**R**	**b**	**r**								
3	3				**3**	**C**	**S**	**c**	**s**								
4	4			**$**	**4**	**D**	**T**	**d**	**t**								
5	5				**5**	**E**	**U**	**e**	**u**								
6	6				**6**	**F**	**V**	**f**	**v**								
7	7				**7**	**G**	**W**	**g**	**w**								
8	8				**8**	**H**	**X**	**h**	**x**								
9	9				**9**	**I**	**Y**	**i**	**y**								
10	A	换行				**J**	**Z**	**j**	**z**								
11	B					**K**		**k**									
12	C					**L**		**l**									
13	D	回车				**M**		**m**									
14	E					**N**		**n**									
15	F					**O**		**o**									

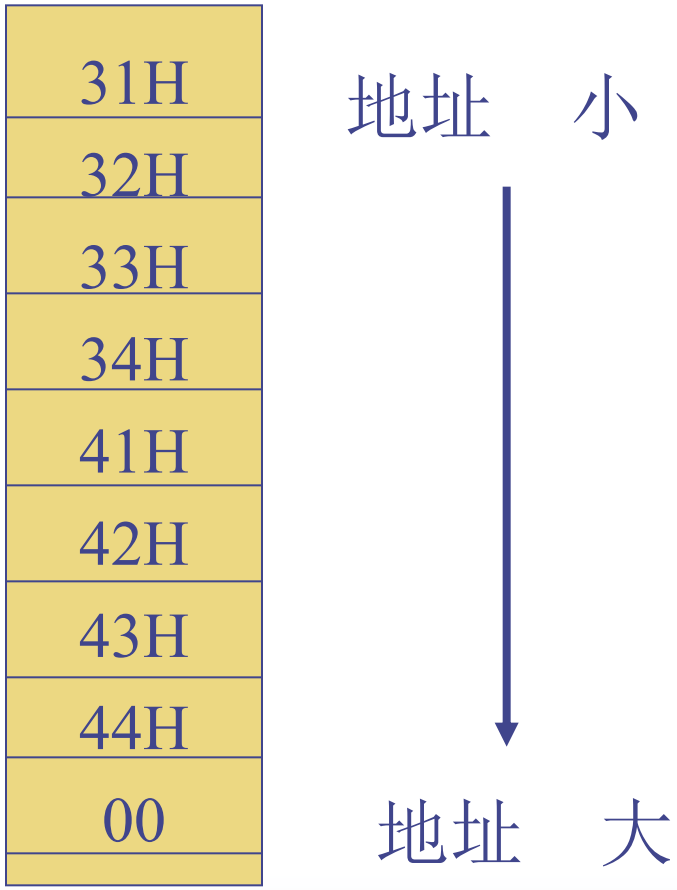
American Standard Code for Information Interchange

2.3 字符数据在机内的表示形式

字符串
“1234ABCD”
的表示结果：

以0 为串的结束

也可以写成’\0’



2.3 字符数据在机内的表示形式

汉字编码 - GB2312

- 简体中文字符集，含6763个汉字和682个非汉字（图形符号），双字节编码。
- 所有字符组成一个94x94的矩阵。矩阵的每一行称为一个“区”，每一列称为一个"位"（行列号称为区位号）。

- 01-09区为682个非汉字字符（图形符号）
- 16-55区为一级汉字（3755个最常用的汉字，按拼音字母顺序排列）
- 56-87区为二级汉字（3008个汉字，按部首次序排列）
- 10-15区、88-94是空白区（保留区）。

2.3 字符数据在机内的表示形式

汉字编码 - GB2312

区位码：由行号（区号）为高字节、列号（位号）位低字节组成的字

国际码（交换码）：区位码 + 0x2020（似乎没什么用）

机内码：区位码 + 0xA0A0

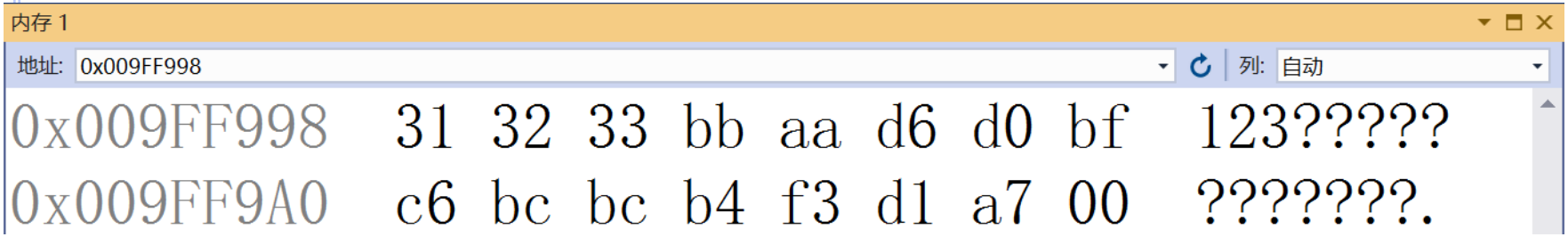
‘华’：区位码 = 27 10，即 0x1B0A

国际码 = 0x3B2A

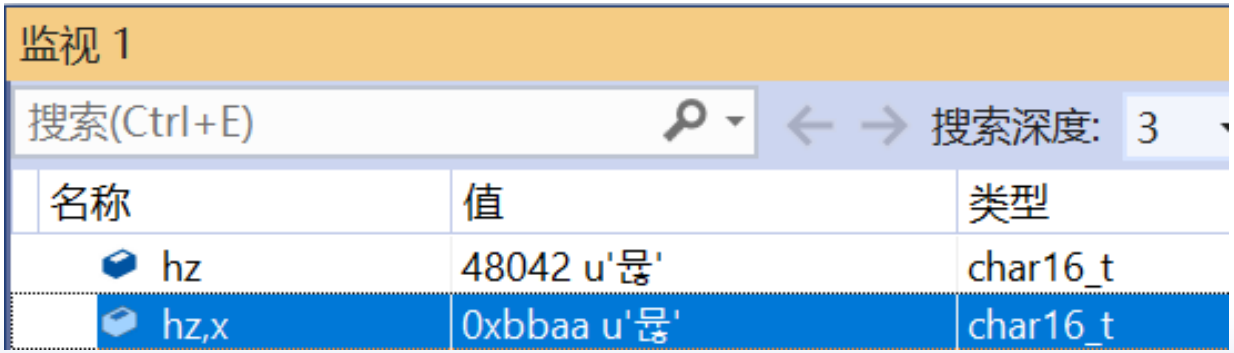
机内码 = 0xBBAA（计算机存储）

2.3 字符数据在机内的表示形式

char s[] = "123华中科技大学";



char16_t hz = '华';



2.3 字符数据在机内的表示形式

Unicode：

- 将全世界所有的字符包含在一个集合里；
- 只要支持该字符集，就能显示所有字符，不会有乱码了。
- 从0开始，为每个符号指定一个编号(码点,code point)

  码点U+0041：英语的大写字母A；码点U+534E：汉字“华”
- 在Unicode 7.0版中，收入了109449个符号，其中的中日韩文字为74500个。
- 最前面的65536个字符位，码点从 U+0000 到 U+FFFF。

  称为基本多文种平面（BMP），含最常见的字符。

  Basic Multilingual Plane

2.3 字符数据在机内的表示形式


```
char16_t   hz  = '华';      //GB2312
char16_t  hz1 = u '华';     //Unicode, UTF-16
```

hz	48042 u'华'	char16_t
hz,x	0xbbaa u'华'	char16_t
hz1,x	0x534e u'华'	char16_t
hz1	21326 u'华'	char16_t

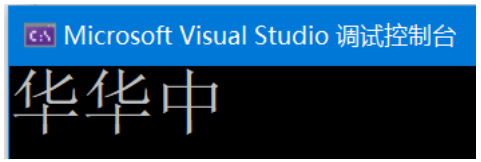
猴:29492 吼:21564 厚:21402 候:20505 后:21518 呼:21628 乎:20046 忽:24573 瑚:29786 壶:22774
葫:33899 胡:32993 蝴:34676 狐:29392 糊:31946 湖:28246 弧:24359 虎:34382 唬:21804 护:25252
互:20114 沪:27818 户:25143 花:33457 哗:21719 华:21326 猾:29502 滑:28369

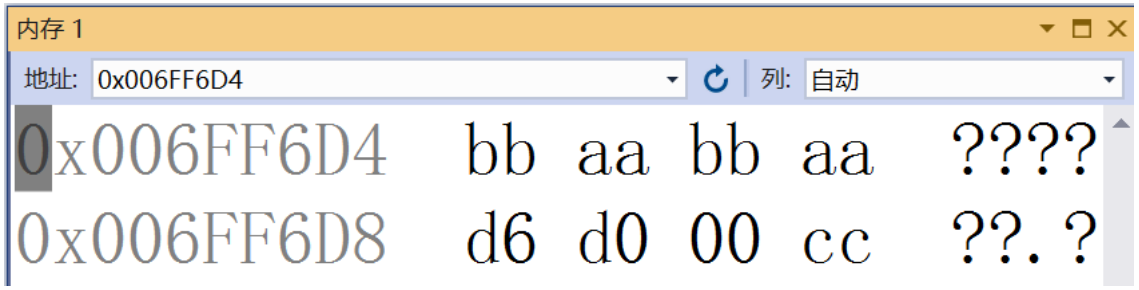
汉字的 unicode 编码表

https://blog.csdn.net/Aimless520/article/details/126692649

2.3 字符数据在机内的表示形式


```
char  p[] = "华\u534e中";
cout << p << endl;
```





2.3 字符数据在机内的表示形式

Unicode存在的问题：

- 如果所有字符都用2个字节、3个字节甚至更多字节来表示，会造成存储空间的增加，太费存储空间
- ASCII只需要一个字节来进行编码
- 如何区别该编码是Unicode还是ASCII ？

- 如何节约空间？
- 如何避免对编码误判？

Unicode 的实现方式：UTF-8、UTF-16、UTF-32

Unicode Transformation Format， 转换格式

2.3 字符数据在机内的表示形式

UTF-8

- 是一种变长的编码方法，长度从1个到4个字节不等；
- 越是常用的字符，字节越短；
- 对于单字节的符号

  字节的第一位设为0，后面7位为这个符号的Unicode码。

  对于英文字母，UTF-8 编码和 ASCII 编码是相同的。
- 对于n字节的符号（n>1）

  第一个字节的前n位都设为1，第n+1位设为0，

  后面字节的前两位一律设为10，

  剩下没有提及的二进制位，全为这个符号的 Unicode 码。

2.3 字符数据在机内的表示形式

- 对于n字节的符号（n>1）

  第一个字节的前n位都设为1，第n+1位设为0，

  后面字节的前两位一律设为10，

  剩下没有提及的二进制位，全为这个符号的 Unicode 码。

  码点U+534E： 汉字“华” → UTF-8

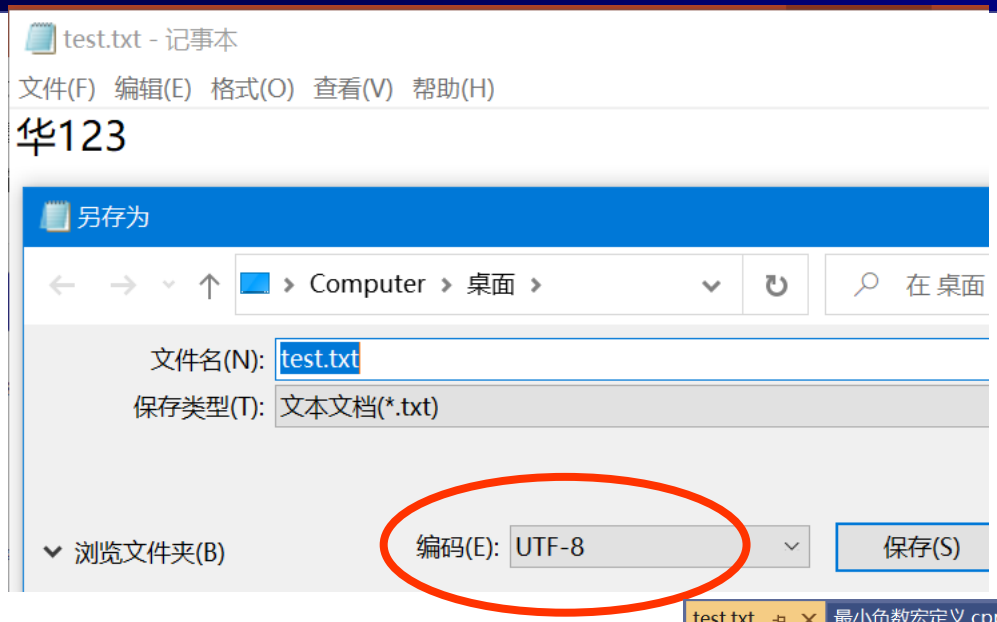
  0101 0011 0100 1110

  15个有效二进制位，用3个字节来表示

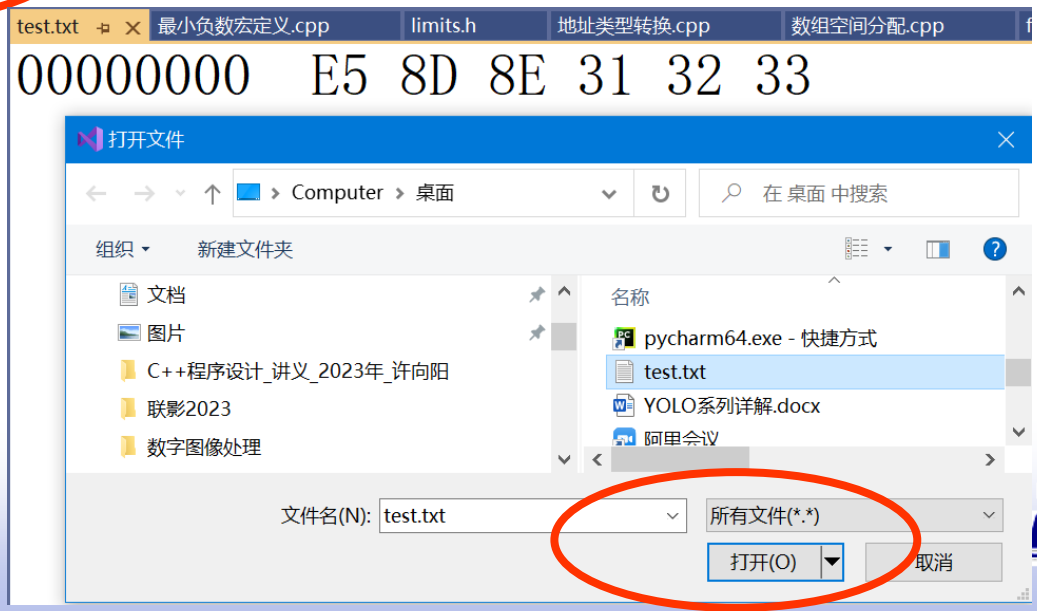
  1110 0101 10 00 11 01 10 00 1110

  E5 8D 8E

2.3 字符数据在机内的表示形式



保存文件时，可以选择编码方式



打开方式中，以2进制形式打开

2.3 字符数据在机内的表示形式

UTF-16

- 介于UTF-8和UTF-32之间，结合了定长和变长两种编码方式的特点；
- 基本平面的字符用2个字节，辅助平面的字符用4个字节。
- 2个字节（U+0000 到 U+FFFF）

  UTF-16编码就是码点 对应的16位无符号整数
- 4个字节（U+010000 到 U+10FFFF）

  先计算 U’ = U - 0x10000

  将U’写成二进制形式：yyyy yyyy yyxx xxxx xxxx，

  U的UTF-16编码：110110yyyyyyyyyy 110111xxxxxxxxxx。

2.3 字符数据在机内的表示形式

UTF-32

- UTF-32是最直观的编码方法
- 每个码点使用四个字节表示
- 字节内容一一对应码点
- 转换规则简单直观，查找效率高。缺点在于浪费空间
- 同样内容的英语文本，它会比ASCII编码大四倍。
- 实际上没有人使用这种编码方法

  HTML 5标准就明文规定，网页不得编码成UTF-32。

2.3 字符数据在机内的表示形式

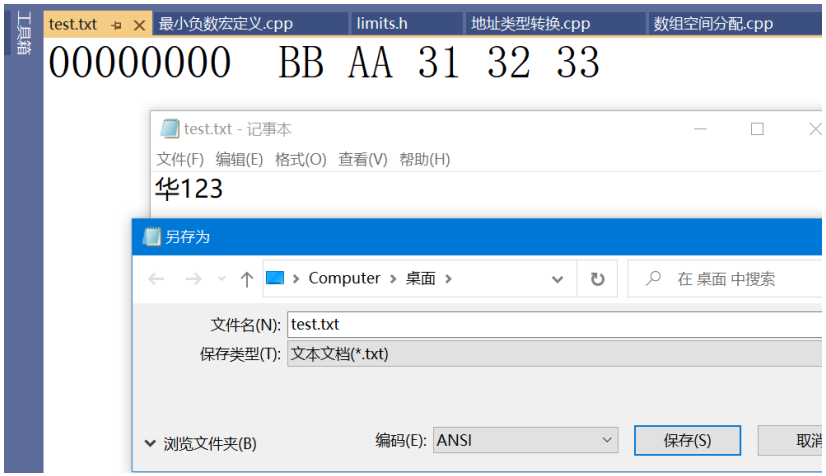
编码(E): GB18030

ANSI 编码

- 不同的国家和地区制定了不同的标准
- 在简体中文Windows操作系统中，ANSI编码代表GB编码;
- 在日文Windows系统中，ANSI编码代表 Shift_JIS 编码。
- 不同 ANSI 编码之间互不兼容;
- 对于ANSI编码而言，0x00～0x7F之间的字符，依旧是1个字节代表1个字符。

American National Standards Institute 美国国家标准学会

2.3 字符数据在机内的表示形式



在 GB2312 中，“华”的区位码是 2710，第27区的第10个字 (共 94区，每区 94个字) (1B 0A)。

区位码的每个字节加 A0，得到 ANSI 编码，即 BB AA。

为了保证 Windows 在不同语言文字的国家都能用，微软采用了标准代码页 (Code Page，代码页是字符集编码的别名，也叫内码表) 的方法，把全世界的编码方式统一编号，在不同的地方采用对应地方的编码方式，微软把它称为 ANSI 编码。

》chcp

活动代码页 936

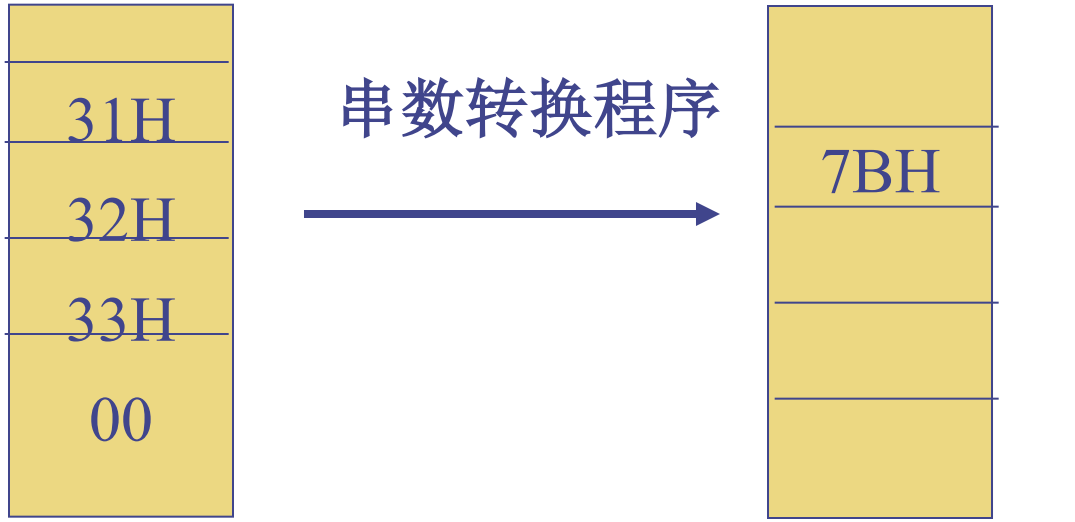
```
C:\WINDOWS\System32>chcp
活动代码页: 936
```

2.3 字符数据在机内的表示形式



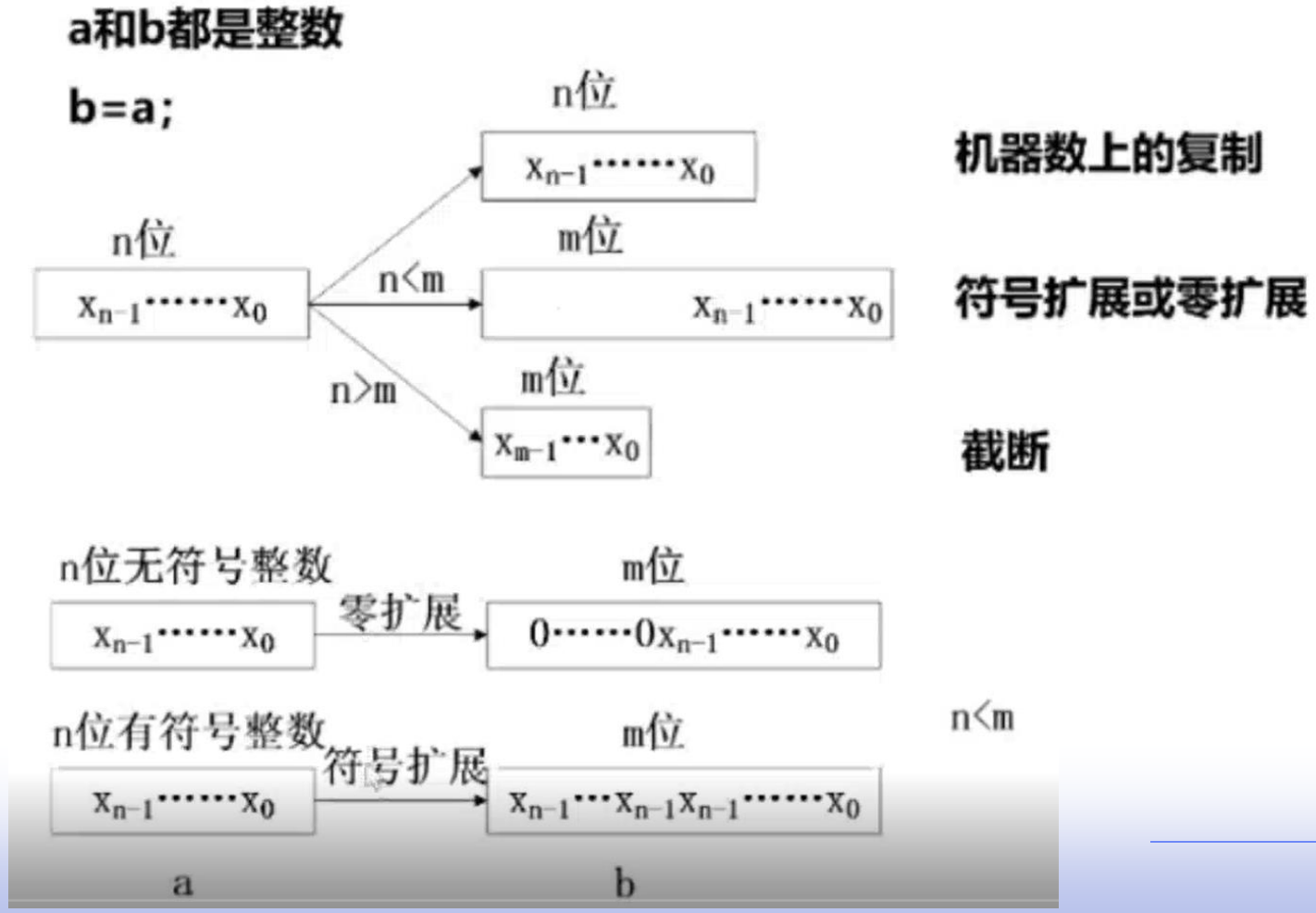
2.3 字符数据在机内的表示形式

Q: 在键盘上输出123。计算机中得到是什么呢？
若要用其作数值运算，如何办呢？



2.4整数类型数据转换

- 不同整数类型的变量赋值，如何进行转换？



2.4整数类型数据转换

- 不同整数类型的变量赋值，如何进行转换？

规则1：两边的长度一样，直接复制，与类型无关；
规则2：右边长，左边短，截断，保留低位；
规则3：右边短，左边长，右边的数需要扩展。

Q：在需要扩展时，何时用符号扩展？
何时用 0 扩展？

```
int  i1;  short s1;  char c1;  bool b1;
unsigned int i2;  unsigned short s2;  unsigned char c2;
```

2.4 整数类型数据转换


```
short s1 = 0xffff;
unsigned short s2 = 0xffff;

int  i1;
i1 = s1;    // i1=0xffffffff;

i1 = s2;    // i1=0x0000ffff;
```

```
movsx  eax,word ptr [s1]
mov    dword ptr [i1], eax
```

```
movzx  eax,word ptr [s1]
mov    dword ptr [i1], eax
```

short → int =>赋值给 int

unsigned short → unsigned int =>赋值给 int

结论：以右边的类型进行扩展

2.4 整数类型数据转换


```
short s1 = 0xffff;                  s1=-1;
unsigned short s2 = 0xffff;         s2=-1;
int  i1;      unsiged int  i2;

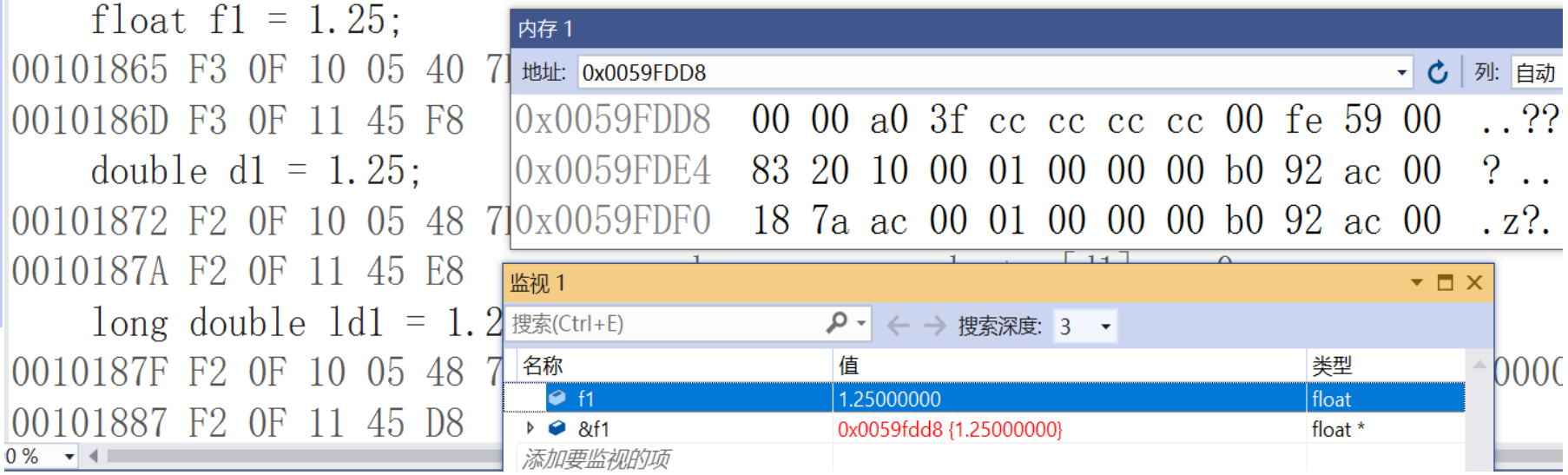
i1 = s1;    // i1=0xffffffff;
i1 = s2;    // i1=0x0000ffff;

i2 = s1;    // i2=0xffffffff;
i2 = s2;    // i2=0x0000ffff;
```

结论：以右边的类型进行扩展；

与 赋值号 左边变量的类型无关

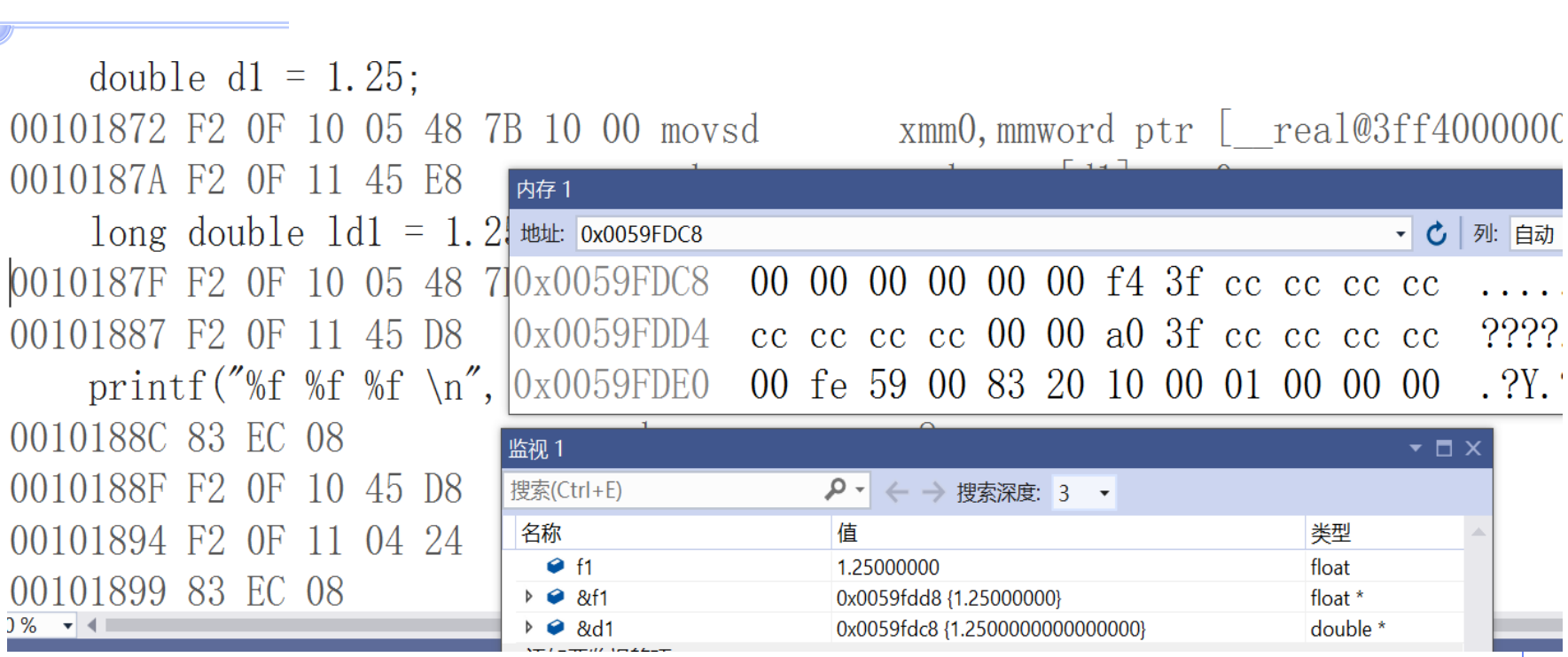
2.5 浮点数据在机内的表示形式



float f1=1.25;

对应的4个字节内容为： 3f a0 00 00

2.5 浮点数据在机内的表示形式



double d1=1.25;

对应的8个字节内容为：3f f4 00 00 00 00 00 00

2.5 浮点数据在机内的表示形式

$$\pm(a_n 2^n + \cdots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + \cdots + a_{-m} 2^{-m} + \cdots)$$

$1.25 = 1.01\text{B}; \quad = 0.101\text{B} * 2 = 0.0101\text{B} * 2^2;$

规格化数据： $\pm 1.\text{XXXX} * 2^n$ B

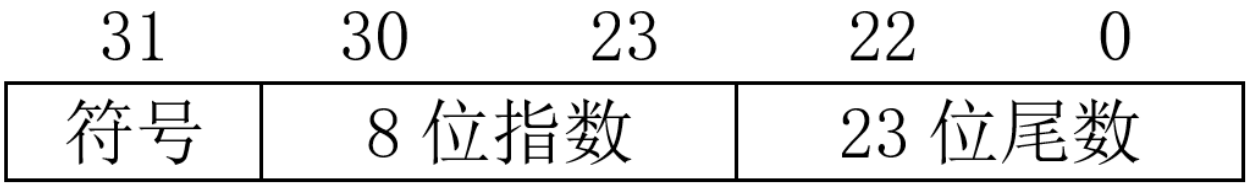
科学表示法：小数点左边恒为1

要素：正负号、尾数 XXXX、指数 n

问题：尾数 XXXX、指数各用多少位来表示？

用什么形式来表达？

2.5 浮点数据在机内的表示形式



单精度浮点数
float

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。单精度移 7F

-126 D -> 00000001 B

127 D -> 11111110 B

尾　数：用原码表示。对于规格化浮点数，尾数的小数点前面的1，不需存贮。

$1.25 = 1.01B = 1.01 * 2^0$;

0 0111 1111 010 00000000000000000000 B

对应的4个字节内容为： 3f a0 00 00

2.5 浮点数据在机内的表示形式



双精度浮点数
double

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。双精度移 3FF

-1022D -> 000 0000 0001B

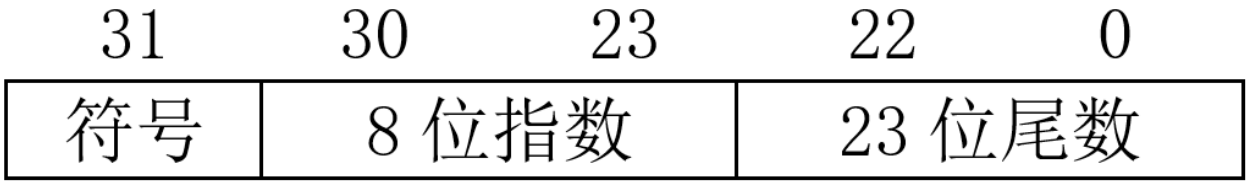
1023D -> 111 1111 1110B

double d1=1.25 = 1.01 * 2^0 ;

0 011 1111 1111 0100 0000 …… 0000

对应的8个字节内容为：3f f4 00 00 00 00 00 00

2.5 浮点数据在机内的表示形式



单精度浮点数
float



双精度浮点数
double



高精度浮点数
long double

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。

单精度移 7F　　双精度移 3FF

高精度移 3FFF

2.5 浮点数据在机内的表示形式

对于float

最小的规格化正浮点数是： $1.00...000B * 2^{-126}$

非规格化数据：阶码为 0

比上面数小一丁点的数是： $0.11....111B * 2^{-126}$

继续逐步变小，最后会有： $0.00....001B * 2^{-126}$

再小一丁点，就是： $0.00 ...000B * 2^{-126}$

0 的表示（阶码为0，尾数为0）。

2.5 浮点数据在机内的表示形式

31	30	23	22	0
符号	8 位指数		23 位尾数	

单精度浮点数：**float**

(1) 规格化浮点数
(2) 非规格化浮点数
(3) 特殊数值（±∞, NaN等）

表 2.2 IEEE 754 浮点数的解释

值的类型	单精度（32 位）				双精度（64 位）			
	符号	阶码	尾数	值	符号	阶码	尾数	值
正零	0	0	0	0	0	0	0	0
负零	1	0	0	−0	1	0	0	−0
正无穷大	0	255(全1)	0	∞	0	2047(全1)	0	∞
负无穷大	1	255(全1)	0	−∞	1	2047(全1)	0	−∞
无定义数（非数）	0 或 1	255(全1)	≠0	NaN	0 或 1	2047(全1)	≠0	NaN
规格化非零正数	0	0 < e < 255	f	$2^{e-127}(1.f)$	0	0 < e < 2047	f	$2^{e-1023}(1.f)$
规格化非零负数	1	0 < e < 255	f	$-2^{e-127}(1.f)$	1	0 < e < 2047	f	$-2^{e-1023}(1.f)$
非规格化正数	0	0	f≠0	$2^{-126}(0.f)$	0	0	f≠0	$2^{-1022}(0.f)$
非规格化负数	1	0	f≠0	$-2^{-126}(0.f)$	1	0	f≠0	$-2^{-1022}(0.f)$

2.5 浮点数据在机内的表示形式

规格化浮点数： $\pm 1.XXXX * 2^n$ B (不存贮整数位1)

n (移码) 范围： [00000001, 11111110]

因此，规格化单精度浮点指数范围：[-126, 127]

尾数范围：[000…000 (23个0), 111…111]

规格化单精度浮点数范围:

$\pm$[$1.0*2^{-126}$, $1.111\ldots111*2^{127}$] B ⇔

$\pm$[$1.175494351*10^{-38}$, $3.402823466*10^{38}$] D

Q：规格化浮点数能表示0和很小的数(如 2^{-130})吗？

2.5 浮点数据在机内的表示形式

最小的规格化正浮点数：$1.0*2^{-126}$

非规格化浮点数： $\pm 0.XXXX * 2^{-126}$ B

规定： 阶码(移码) = **00000000** （尾数不为0）

非规格化单精度浮点数范围:

$\pm$[0.000…001*2^{-126}, 0.111…111*2^{-126}] B ⇔

$\pm$[1.4013*10^{-45}, 1.17548*10^{-38}] D

对于区间(-1.4013*10^{-45}, 0)、(0, 1.4013*10^{-45})的数，float是无法表示的 !!!

2.5 浮点数据在机内的表示形式

特殊数值

+∞ 符号位0、阶码全1、尾数为0

0-1111111 1-0000000 00000000 00000000

-∞ 符号位1、阶码全1、尾数为0

1-1111111 1-0000000 00000000 00000000

NaN (Not a Number) 符号位1或0、阶码全1、尾数不为0

例如：0-1111111 1-0000000 00000000 00000001

一些因素会引起 NaN（非数值），如 0 / 0、sqrt(-1) 等。

+0 符号位0、阶码全0、尾数为0

0-0000000 0-0000000 00000000 00000000

-0 符号位1、阶码全0、尾数为0

1-0000000 0-0000000 00000000 00000000

2.5 浮点数据在机内的表示形式

Problem1:

```
float  a = 10000;
float  b = a + 0.0001f;
bool  c1 = (a == b);
bool  c2 = (10000.0f < 10000.0001f);
```

c1= true
c2 = false

c1, c2 = true or false ?

Problem2:

```
float  a = 0x65536;
float  b;
```

求满足 b > a 的最小 b。

a= 415030
48 ca a6 c0
b = 48 ca a6 c1
415030.031

“Father” of the IEEE 754 standard

80年代初，各个机器内部的浮点数表示格式还没有统一

相互不兼容，机器之间传送数据时，带来麻烦

1970年代后期，IEEE成立委员会着手制定浮点数标准

1985年完成浮点数标准IEEE 754的制定

This standard was primarily the work of one person, UC Berkeley math professor William Kahan.

1989
ACM Turing
Award Winner!

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html



Prof. William Kahan

2.5 浮点数据在机内的表示形式

Q：为什么指数使用移码表示，而不是补码表示？

在加法运算时，要对阶，用无符号数比较、移位更简单。

Q：为什么规格化时，用 $\pm 1.XXXX * 2^n$ B ，

而不是 $\pm 0.1XXXX * 2^n$ B ？

能够保存更多的有效位数，指数的表示范围也更大。

2.6 整数与浮点类型数据转换

同样一个数值，分别用 short, int, float, double 表示，结果相同吗？

```
int  i;  float  f;  double  d;
```

Q: i == (int)(float)i; ?

```
#include <stdio.h>

int main()
{
    int  i;
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(float)i;
    printf("%d \n", i);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
1234567890
1234567936
```

2.6整数与浮点类型数据转换

同样一个数值，分别用 short,int,float,double 表示，结果相同吗？

```
int  i;  float  f;  double  d;
```

Q: i == (int)(double)i; ?

```
int main()
{
    int  i;
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(float)i;
    printf("%d \n", i);
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(double)i;
    printf("%d \n", i);
    return 0;
```



float 23 位尾数

double 52位尾数

2.6整数与浮点类型数据转换

同样一个数值，分别用 short,int,float,double 表示，结果相同吗？

```
int  i;  float  f;  double  d;
```

Q: f == (float)(int)f; ?

总结

有符号整数和无符号整数的表示与存储

字符串的表示方法与存储

浮点数据的表示方法与存储

数据类型转换方法